IR LED Pulsing — Implementation & Console Handout

Oct 29, 2026 · @Ramkrishna

Overview

The STM32H563 proxy MCU sets the IR LED current on an LT3763 driver and switches the LED with an inverted gate tied to the sensor's FLASH signal. Only CH0 (J5) has an LED board fitted. The firmware is on branch `ir-20a-pulse` @ `fdf8e30`, IMX568 build, plus the flash PWM mode added on 2026-10-29 (not yet committed).

Hardware per channel (proxy J5/J6 map pin-for-pin onto the USHIO board's J3):

J5/J6 pin	Signal	MCU pin (CH0 / CH1)	Goes to
1	DAC_DIMx	PA4 / PA5 (DAC1 ch1/ch2)	LT3763 CTRL1, sets the current
2	IR_FAULTx	PB9 / PE0 (input)	LT3763 FAULT, open drain, active low
3	IR_CTRLx	from PWM_DIMx PB14 / PB15 via Q3/Q2	LT3763 PWM, turns the LED on and off
4 / 5	3V3 / GND	—	—

The gate is inverted. PWM_DIMx drives an N-FET whose source sits on the FLASH net (PC3):

- PWM_DIMx **LOW** → FET off → IR_CTRLx pulled to 3V3 → **LED ON**.
- PWM_DIMx **HIGH** → FET on → IR_CTRLx follows FLASH → LED on only while FLASH is high.

The MCU can force the LED on at any time, but can turn it off only while FLASH is low.

Current path. The LT3763 regulates the sense voltage to the lowest of its CTRL pins: 0–1.5 V on CTRL1 maps to 0–51 mV across R11 = 2.5 mΩ, i.e. 0–20.4 A. R16 was reworked to 0R on 2026-10-13, so CTRL2 = 2.0 V no longer clamps the board at 4.92 A. The firmware limits how far the current can actually go (see Current limits).

LED. USHIO SMBB850DS-1200, rated IF = 4 A DC and IFP = 20 A pulsed.

Not available to the MCU: ISMON/IVINMON (actual LED current) go only to test points TP1/TP2, so every current the firmware reports is a setpoint, open loop. The LED board has no enable pin; it runs whenever the 18 V rail is up.

Console: UART4 (PA0 TX / PA1 RX), 115200 8N1. On the bench it's the CP2105 second port, `/dev/ttyUSB1`.

Pulse engine

Every pulse mode above 4 A works the same way: raise DAC_DIMx to the pulse current, open the gate on a sync edge, and close the gate and drop the DAC to 0 V on the opposite edge. A hardware timer is a backstop that ends the pulse if that edge never arrives.

Building blocks

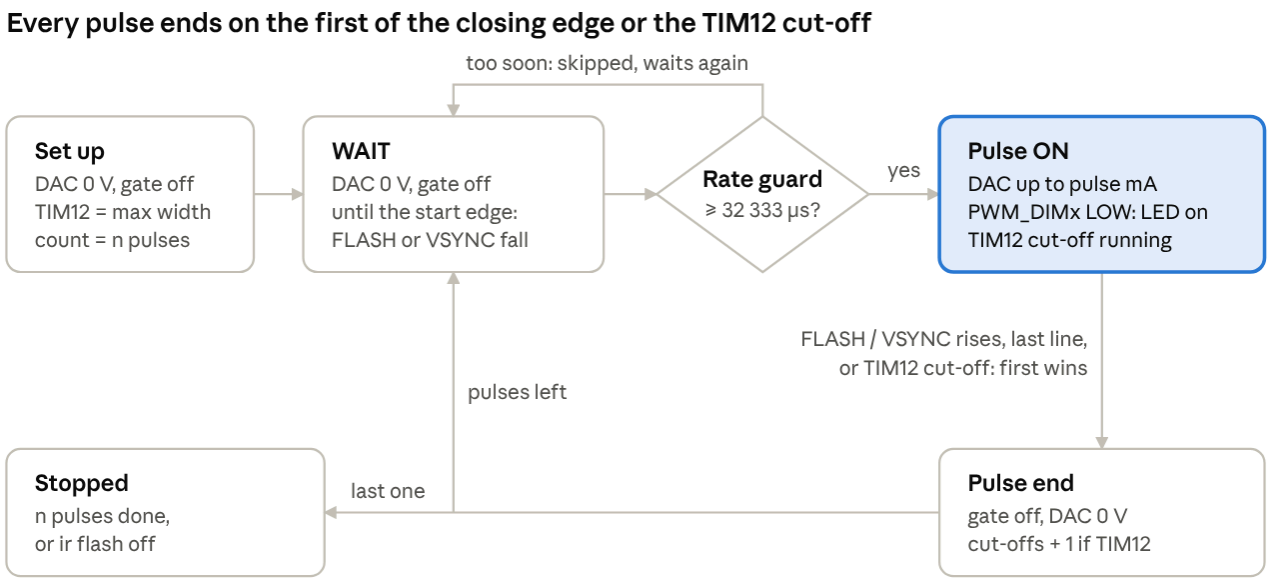
Block	Role
DAC1 ch1/ch2 (PA4/PA5)	Pulse current on CTRL1. Rounded **down** so a pulse never exceeds the requested mA (about 11 mA per DAC code with R16 = 0R).
PWM_DIMx (PB14/PB15)	Gate. LOW forces the LED on.
EXTI on PC3 (FLASH), PC4 (SN0_VS), PB1 (SN0_HS)	Sync edges. They start and end the pulse in the interrupt.
TIM12, one-pulse mode, 1 µs tick, 16-bit	**Cut-off.** Started when each pulse begins, with ARR = the derating width; its update interrupt (priority 0, highest) closes the gate and sets the DAC to 0 V. Maximum 65 535 µs.
TIM7	30 Hz frame tick for `hpulse` and the frame pulse when no VSYNC is present.
DWT cycle counter	Edge timing, FLASH LOW width and the rate guard.

Rate guard. A pulse that starts less than `IR_HP_MIN_SPACING_US` = 33 333 − 1 000 = **32 333 µs** after the previous one is skipped and counted under `skipped`. A glitching sync input therefore cannot push the pulse rate above 30 Hz, which the derating assumes.

One engine, one channel. The engine and TIM12 are shared, so only one of `hsync`, `fhsync`, `vsync` and `flash` runs at a time, on one channel. Starting one stops the others, as well as `hpulse`, the frame pulse and gate PWM on both channels. On the chosen

channel it also stops ramp, DAC PWM, strobe and the FLASH gate. The DC setpoint goes to 0, and PWM_DIMx stays HIGH between pulses.

Outside a pulse the DAC is always 0 V. This matters because while FLASH is high, Q2/Q3 pass it to the LT3763 PWM pin whatever PWM_DIMx does. A DAC left at 20 A would light the LED for most of every frame.



pulse engine (hsp_*) in ir_led_driver.c · flash and vsync modes

The `hsync` modes add an ARMED state between WAIT and the pulse: VSYNC arms the engine, and the HSYNC line count starts the pulse at `<line>`. Solid line pulses also add a PRE line, during which the DAC is up but the gate is still closed.

In `flash ... pwm`, Pulse ON hands PWM_DIMx to TIM12 hardware PWM instead of holding it LOW. Each TIM12 update then counts one PWM period instead of cutting the pulse. The burst ends when the budget's periods are used up or FLASH rises, whichever comes first.

Pulsing modes

On the IMX568 board, use **`flash`**: it is the only synced pulse mode that board can run, because the IMX568 outputs no XVS/XHS (SN0_VS and SN0_HS counts stay at 0). The VSYNC/HSYNC modes are for a sensor that provides those signals, such as the IMX678.

Pulse modes (up to 20 A, derating enforced, on the shared pulse engine or TIM12)

Mode	LED is on	Sync source	Ends a pulse	IMX568	IMX678
`flash`	While FLASH (PC3) is LOW, i.e. the exposure	FLASH falling / rising edge	FLASH rise, or TIM12 at the derating width	**Yes**, boot default	Refused

Mode	LED is on	Sync source	Ends a pulse	IMX568	IMX678
flash <mA> pwm	In 1 kHz PWM pulses through FLASH LOW, spreading the per-frame budget over the whole exposure (solid if the budget covers it)	FLASH falling / rising edge	FLASH rise, or the last PWM period the budget allows (TIM12 update count)	Yes (new; counters verified 2026-10-29)	Refused
`hpulse`	For a fixed width once per frame	VSYNC; without it, TIM7 free-running at 30 Hz	TIM12 one-pulse after `<us>`	Yes, unsynced (drifts against the sensor)	Yes
`vsync <mA>`	While SN0_VS (PC4) is LOW	VSYNC falling / rising	VSYNC rise, or TIM12	No VSYNC	Yes
`vsync <mA> <line> <lines>`	Solid, from `<line>` for `<lines>` lines after VSYNC	VSYNC + HSYNC line count	Line count, or TIM12	No	Yes
`hsync`	During each HSYNC HIGH inside the line window	HSYNC (PB1) after VSYNC	Each HSYNC fall; TIM12 ends the window	No	Yes
`hsync ... inv`	During each ~1.7 µs HSYNC LOW inside the window	HSYNC	Next HSYNC rise; TIM12	No	Yes
`fhsync`	As `hsync`, but the gate is PC3 itself, driven as an output; PWM_DIMx stays HIGH	HSYNC	As `hsync`	Only with the sensor's FLASH output **disabled**	Yes (PC3 is an output there)

Gate and current modes (4 A cap, no derating needed)

Mode	What it does	Notes
`gate on`	PWM_DIMx LOW while FLASH is low, HIGH when it rises	CH0 at boot. Only one channel may be gated.
`gate inv`	PWM_DIMx HIGH while FLASH is low, LOW while high	LED off during the exposure; for testing the gate polarity.
`strobe`	FLASH gate **plus** DAC at the setpoint while FLASH is low, 0 V while high	Dim, not dark, between exposures: the LT3763 offset can still pass up to ~1.2 A at 0 V.

Mode	What it does	Notes
`pwm high` / `pwm low`	PWM_DIMx fixed HIGH (LED follows FLASH) / fixed LOW (LED forced on)	`low` refused while the other channel is gated.
`pwm <hz> <duty%>`	TIM12 hardware PWM on PWM_DIMx; duty = LED-on share	10–20 000 Hz, frequency shared by both channels.
`ma` / `mv` / `ramp`	DC current setpoint, a DAC voltage, or a non-blocking linear ramp (1 DAC step per ms)	Ramp time 0–600 000 ms.
`dacpwm <hz> <duty%>`	Chops DAC_DIMx between 0 V and the setpoint	1–10 000 Hz. The LT3763 loop rounds the edges off; this is not true PWM dimming.
`dactest <mV>`	DAC voltage 0–1500 mV, **bypassing the 4 A cap**	Bench only, with J5/J6 **unplugged**: 1500 mV on a connected driver is ~20 A DC.

Rules that apply across the modes

- Only one channel can be FLASH-gated, and gating is refused while the other channel's PWM_DIMx is held LOW. FLASH is pulled high only through a channel whose PWM_DIMx is HIGH, so with both channels low FLASH never rises and the LEDs latch on.
- TIM12 has one frequency and one period, so `pwm`, `hpulse` and the frame pulse share them across both channels, and every pulse-engine mode takes TIM12 for itself.

Console commands

Connect at 115200 8N1 on UART4 (for example `picocom -b 115200 /dev/ttyUSB1`) and type commands at the `MCU>` prompt. `[ch0|ch1]` is optional and defaults to **ch0**. `[n]` is a pulse or frame count; 0 or omitted means continuous.

Pulse commands

Command	Effect	`on` repeats
`ir [ch] flash <mA> [n]`	LED at `<mA>` (≤ 20 000) while FLASH is LOW; TIM12 cut-off at the derating width	last settings, default `100`

Command	Effect	`on` repeats
ir [ch] flash <mA> [n] pwm	As flash, but PWM_DIMx is chopped by TIM12 at 1 kHz so the per-frame budget covers the whole exposure	last settings, pwm included
`ir [ch] flash on` / `off`	Repeat the last settings / stop (DAC 0 V, PWM_DIMx HIGH)	
`ir [ch] hpulse <mA> <us> [n]`	One pulse of `<us>` per 30 Hz frame; width clamped to the derating	—
`ir [ch] hpulse off`	Stop; DAC 0 V	
`ir hpulse table`	Print the maximum width per current at 30 Hz	
`ir [ch] hsync <mA> <line> <lines> [n] [inv]`	Gate follows HSYNC HIGH (or LOW with `inv`) for `<lines>` lines from `<line>` after VSYNC	default `20000 100 21`
`ir [ch] fhsync <mA> <line> <lines> [n] [inv]`	As `hsync`, but PC3 is driven as the gate; `off` returns PC3 to an input	last settings
`ir [ch] vsync <mA> [n]`	LED on while VSYNC is LOW	default `20000`
`ir [ch] vsync <mA> <line> <lines> [n]`	One solid pulse per frame over the line window	
`ir hsmeas`	Measure HSYNC timing (diagnostic)	

Current and gate commands

Command	Effect
`ir [ch] ma <mA>`	DC setpoint, 0–4000 mA
`ir [ch] mv <mV>`	DAC_DIMx voltage, clamped at the 4 A code (~294 mV)
`ir [ch] ramp <mA> <ms>`	Linear ramp to `<mA>` over 0–600 000 ms, non-blocking
`ir [ch] strobe on` / `off`	FLASH gate plus DAC switching

Command	Effect
`ir [ch] gate on` / `off` / `inv`	PWM_DIMx follows FLASH / fixed HIGH / inverted
`ir [ch] pwm high` / `low` / `<hz> <duty%>`	PWM_DIMx fixed or TIM12 PWM (10–20 000 Hz)
`ir [ch] dacpwm <hz> <duty%>` / `off`	DAC chop, 1–10 000 Hz
`ir [ch] dactest <0..1500 mV>` / `off`	DAC test that bypasses the 4 A cap; **J5/J6 unplugged**
`ir cut day` / `night`	Pulse the IR-cut filter in / out (PA11 EN, PA12 PH)
`ir cut ms <10..1000>`	IR-cut pulse length, default 150 ms
`ir cut swap`	Swap PH polarity if day and night come out reversed
`ir status`	Inputs, both channels, active mode, counters, IR-cut
`ir help`	Full command list

Example session (IMX568, sensor streaming):

```
MCU> ir flash 5000
SUCCESS: ch0 FLASH pulse 4995 mA while FLASH low, cut-off 17602 us,
continuous
MCU> ir flash off
MCU> ir hpulse 20000 300 10      # ten 300 us pulses at 20 A, one per frame
MCU> ir flash on                 # back to the last flash settings
```

The currents in the example's responses are illustrative: the firmware rounds each current down to the nearest DAC step.

Reading `ir status`

Line	What it tells you
`FLASH (PC3 in): HIGH 30.00 Hz low 13.33 ms falls N`	The sensor is streaming; LOW time = exposure = LED pulse length
`FLASH ... NOT toggling (no edge for X ms)`	No edge for 200 ms or more; `flash` will not start
`SN0_VS` / `SN0_HS ... NOT toggling`	Normal on the IMX568

Line	What it tells you
`DAC_DIM0: X mV (code C) -> ~I mA`	The DAC right now: 0 V between pulses in pulse modes
`FLASH PULSE: I mA while FLASH LOW, cut-off W us`	`flash` is running
`fired / timer cut-offs / skipped`	Pulses started / ended by TIM12 (FLASH low longer than the cut-off) / refused by the rate guard
`FAULT0 (PB9): high`	No fault. Low is meaningful only with the gate on for 50 ms or more, and FAULT1 is meaningless because CH1 has no driver.

Current limits and derating

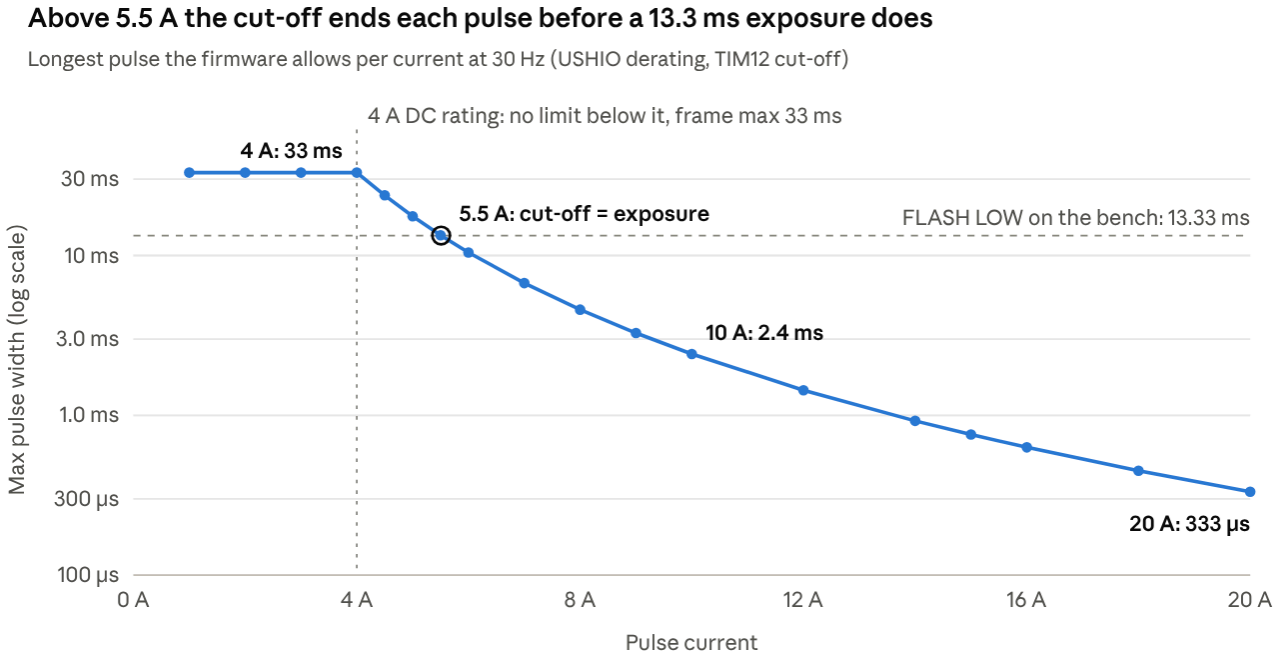
Up to 4 A the LED can be lit for the whole 33 ms frame. Above 4 A, only short pulses are allowed, capped at 20 A for 333 µs.

Limit	Value	Applies to	Enforced by
Hard DC cap `IR_LED_HARD_LIMIT_MA`	**4000 mA** (~294 mV on CTRL1)	`ma`, `mv`, `ramp`, `strobe`, `gate`, `dacpwm`; no runtime override	Clamp on the DAC code in `IR_LED_SetCurrentRaw()`
Pulse cap `IR_LED_PULSE_LIMIT_MA`	**20 000 mA**	`flash`, `hpulse`, `hsync`, `fhsync`, `vsync`	Clamped with a console NOTE
Pulse width above 4 A	derating curve below	the same pulse modes	TIM12 cut-off (`flash`, `vsync`); width or line clamp (`hpulse`, `hsync`)
Pulse rate	⩽ 30 Hz (starts ⩾ 32 333 µs apart)	all pulse-engine modes	Rate guard

Limit	Value	Applies to	Enforced by
Board maximum	20.4 A (51 mV / 2.5 mΩ)	hardware, CTRL1 = 1.5 V	LT3763
`dactest`	0–1500 mV, **no current cap**	bench, J5/J6 unplugged	—

The width limit follows the USHIO SMBB850DS-1200 pulse-duration curve: 20 A at 1 % duty, a straight log-log line down to 4 A at 100 %, at f = 30 Hz:

```
duty_max(I) = 1 % × (20 A / I) ^ (ln 100 / ln 5)        for 4 A < I ⩽ 20 A
w_max(I)    = duty_max(I) × 33 333 µs                   (one 30 Hz frame)
```



IR_LED_HighPulseMaxWidth_us() at 30 Hz, ir-20a-pulse @ fdf8e30 · exposure from ir status on 2026-10-29

The bench sensor holds FLASH low for 13.33 ms. Above about 5.5 A, TIM12 therefore ends each pulse before FLASH rises: the LED lights only the first part of the exposure, and `timer cut-offs` counts up in `ir status`. Run `ir hpulse table` on the console for the exact width at any current.

PWM mode above 5.5 A

`ir flash <mA> pwm` keeps the same per-frame on-time budget B as the single pulse, but spreads it across the whole exposure instead of lighting only its start. At each FLASH fall the firmware takes the last measured FLASH LOW time T and runs TIM12 hardware PWM

on PWM_DIMx: period 1000 µs, on-time B × 1000 µs / T.

Pulse current	Budget B per frame	On-time per 1 ms period (duty)	Average current over a 13.33 ms exposure
⩽ 5.5 A	⩾ 13.4 ms	solid, no chopping	up to 5.5 A
6 A	10.4 ms	784 µs (78 %)	4.7 A
8 A	4.6 ms	344 µs (34 %)	2.8 A
10 A	2.4 ms	182 µs (18 %)	1.8 A
15 A	0.76 ms	57 µs (5.7 %)	0.85 A
20 A	0.33 ms	25 µs (2.5 %)	0.5 A

- **The budget is a hard limit.** The TIM12 update interrupt counts periods. After B / on-time of them the burst ends even if FLASH stays low, and CCR is preloaded to 0 one period early so no partial pulse follows the last.
- **Minimum on-time 10 µs** (`IR_FLP_MIN_ON_US`), so the LT3763 reaches the set current. If the calculated on-time is shorter, the period is lengthened instead, keeping the same duty.
- **Dark between PWM pulses.** FLASH is LOW and PWM_DIMx is HIGH, so Q2/Q3 hold IR_CTRLx low. The DAC stays at the pulse current through the burst and drops to 0 V when it ends.
- **It uses the previous frame's exposure length.** If the exposure suddenly gets longer, the budget runs out before FLASH rises and the rest of that exposure is dark. If it gets shorter, part of the budget goes unused.
- **Above 5.5 A the exposure gets less light than at 5.5 A solid,** because the average current falls (see the table). PWM pays off only for exposures shorter than about 2 ms. For the bench's 13.33 ms exposure, `ir flash 5500` gives the most light.
- **Status:** flashed and run on the board on 2026-10-29. Over 39 frames each at 6 A, 10 A and 20 A it gave 784 / 181 / 24 µs per 1 ms, 13 periods per burst (10.2 / 2.35 / 0.31 ms, all within budget), 0 skipped, FAULT0 high. Every burst ended on the budget, about 0.3 ms before FLASH rose. The PB14 waveform and LED current have not yet been checked on a scope.

Boot behaviour and timeouts

After reset the IMX568 build gates CH0 on FLASH at 100 mA. Once FLASH is toggling (2 s after boot at the earliest), it switches to the `flash` pulse at 100 mA.

Boot sequence (IMX568 build)

1. Before `MX_GPIO_Init()` runs, PB14/PB15 are inputs, so IR_CTRLx is pulled to 3V3 and the LED is commanded on. `MX_GPIO_Init()` therefore writes PWM_DIM0/1 HIGH before making them outputs.
2. `IR_LED_Init()`: DAC up, both setpoints 0, PC3 left an input, then CH0 set to `IR_BOOT_CURRENT_MA` = **100 mA** with the FLASH gate on (`IR_FLASH_GATE_AT_BOOT`). CH1 stays at 0 mA.
3. In this state, the LED runs at 100 mA DC during the exposure (gate) and also while FLASH is high (Q2/Q3 pass FLASH).
4. From **2000 ms** after boot (`IR_BOOT_FLASH_PULSE_DELAY_MS`), `IR_LED_Task()` tries every main-loop pass to start `flash` on CH0 at `IR_BOOT_FLASH_PULSE_MA` = **100 mA**. It succeeds once FLASH has had an edge in the last 100 ms.
5. It starts only once per boot. After a `flash off`, or once you pick any other pulse-engine mode, it stays off until the next reset.

The IMX678 build skips steps 4–5: `IR_BOOT_FLASH_PULSE_MA` is 0, and PC3 is an output held LOW from boot.

Timeouts and timing constants

Constant	Value	What happens
TIM12 cut-off (`flash`, `vsync`)	derating width, 333 µs at 20 A up to **33 000 µs** at ⩽ 4 A, 16-bit limit 65 535 µs	Gate closed and DAC set to 0 V if the closing edge has not arrived; counted as `timer cut-offs`
TIM12 cut-off (`hsync`, `fhsync`, line `vsync`)	lines × measured line period + 3 µs (+ 1 pre-line for solid)	Window ends even if HSYNC edges go missing
Rate guard `IR_HP_MIN_SPACING_US`	**32 333 µs**	Earlier starts skipped; counted as `skipped`
`IR_FRAME_PULSE_MAX_US`	33 000 µs	Longest `hpulse` / frame pulse; a pulse always ends before the next frame
FLASH / VSYNC presence for `flash` / `vsync`	edge within **100 ms**	Command refused ("FLASH (PC3) not toggling")
HSYNC presence for `hsync` / `fhsync`	edge within **1 ms**	Command refused

Constant	Value	What happens
`IR_FLASH_IDLE_MS`	200 ms	`ir status` reports FLASH or VSYNC as NOT toggling
`IR_VSYNC_TIMEOUT_MS`	100 ms	`hpulse` / frame pulse fall back to TIM7 at 30 Hz
`IR_FAULT_BLANKING_MS`	50 ms	FAULT is latched only if still low after the gate has been on this long
`IR_CUT_PULSE_MS`	150 ms (10–1000)	IR-cut DRV8210 drive time per `ir cut`
`IR_FLASH_MAX_ON_MS`	40 ms	FLASH ADC sync mode only, which is compiled out (`IR_FLASH_SYNC_ENABLE` = 0)

The engine stops a pulse on the first of three events: the sync edge closing it, the TIM12 cut-off, or the pulse count running out.

Assumptions and limitations

Assumptions the safety limits rely on

- **30 fps.** The derating and the 32 333 µs rate guard assume at most one pulse per 33.3 ms. A sensor running faster is not detected; its extra pulses are just skipped.
- **The datasheet curve.** The USHIO pulse-duration curve is modelled as a straight log-log line, applied at 100 % (`IR_HP_DERATE_PCT`), with no extra margin.
- **R16 = 0R, R11 = 2.5 mΩ.** The current scaling (20.4 A full scale, about 11 mA per DAC code) assumes this rework. A board with the original R16 = 45.3k clamps at 4.92 A, and `IR_CTRL2_R_TOP_OHMS` must be set back.
- **FLASH is active LOW** (nFLASH from the sensor board's Q1). A sensor wired the other way inverts every FLASH mode.
- **IMX568 = FLASH only.** It drives no XVS/XHS, so FLASH is its only sync signal.

Limitations

- **Open loop.** No current readback: ISMON is on TP2 only. Every reported mA is a setpoint after DAC rounding.
- **0 mA is not dark.** At CTRL1 = 0 V the LT3763 current-amp offset can still pass up to ~1.2 A. The gate (PWM_DIMx) and the FLASH level are the only real off switches.

- **The MCU cannot force the LED off while FLASH is high.** Q2/Q3 pass FLASH to the LT3763 PWM pin, which is why every pulse mode drops the DAC to 0 V between pulses.
- **CH1 has no LED driver.** Its DAC stays at 0 and FAULT1 means nothing. Only CH0 (J5) is real.
- **FAULT0 is a status line, not an edge.** It is asserted at every startup and whenever the inductor current is below 10 %, so a low FAULT0 does not mean the LED is dark.
- **One pulse engine.** Only one synced pulse mode, on one channel, at a time.
- **TIM12 is 16-bit at 1 µs**, so no cut-off can exceed 65 535 µs.
- **`hpulse` without VSYNC drifts.** TIM7's 30 Hz is free-running, so on the IMX568 the pulses slide slowly across the exposure. Use `flash` there instead.
- **`fhsync` takes over PC3.** While it runs, PC3 is a push-pull output, which is safe only with the sensor's FLASH output disabled. It is refused while the FLASH ADC sync owns PC3.
- **`dactest` bypasses every cap.** Above ~294 mV it exceeds the LED's 4 A DC rating, and 1500 mV is ~20 A DC. Use it only with J5/J6 unplugged.
- **The DAC can exceed the CTRL1 absolute maximum.** Full-scale DAC is 3.3 V against the LT3763's 3.0 V absolute maximum. The console warns above 3000 mV.

Open issues

- One unexplained BOR+PIN reset happened during a 20 A run. It may be a brown-out on the supply; not confirmed.
- The `ir help` text for `flash on` used to say "default 1000"; fixed on 2026-10-29 to the actual default, 100 mA.
- USART11 (PA6/PA7) is pin-muxed but never initialised; the console is UART4 only.